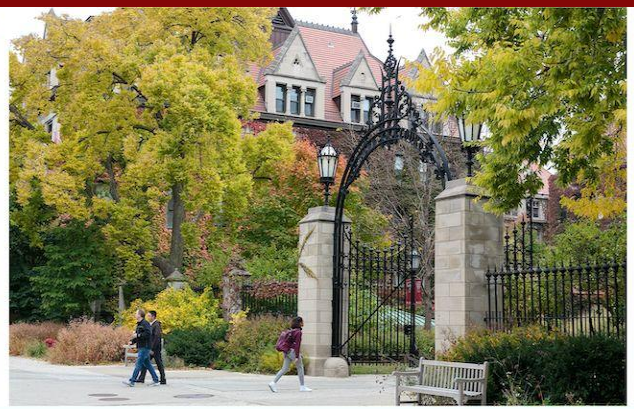
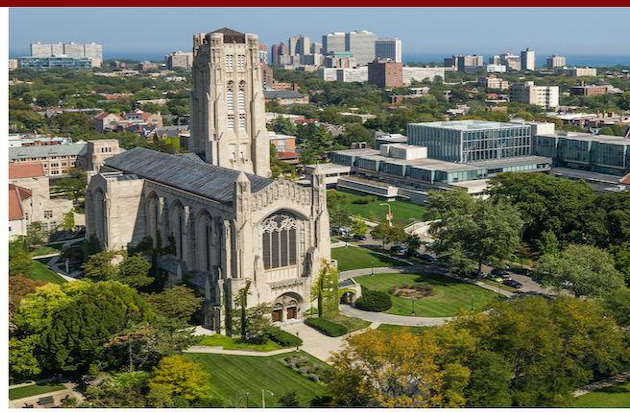
MPCS 51300 - Compilers
M5: Code Generation

Lamont Samuels

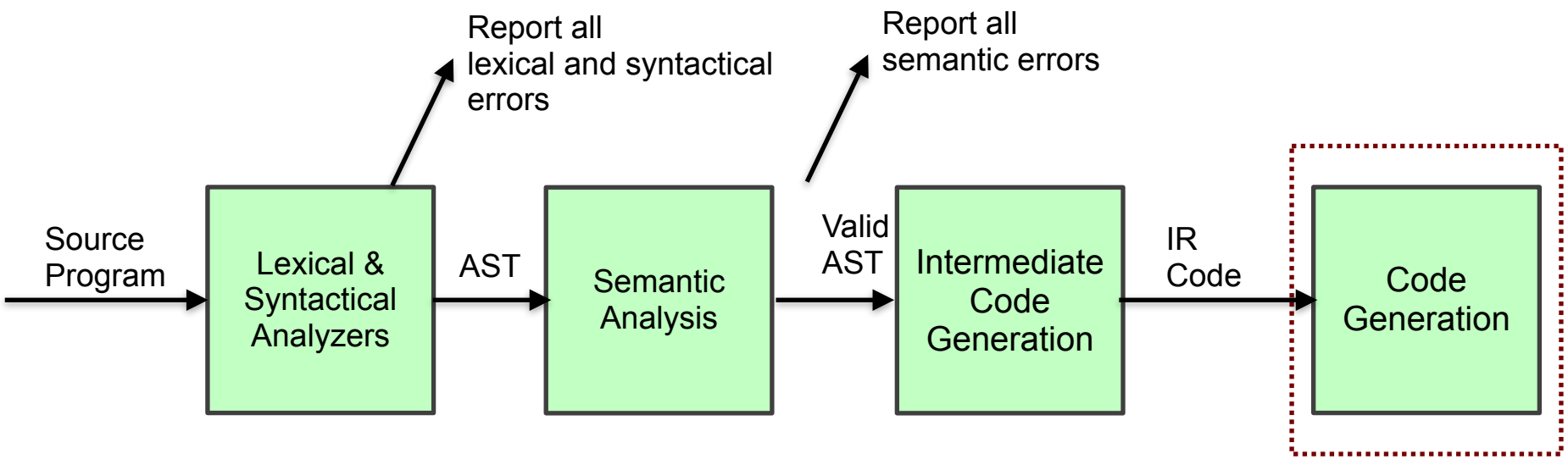
Copyright 2010, Keith D. Cooper & Linda Torczon, all rights reserved.
Students enrolled in Comp 412 at Rice University have explicit permission to make copies of these materials for their personal use.
Faculty from other educational institutions may use these materials for nonprofit educational purposes, provided this copyright notice is preserved.

THE UNIVERSITY OF CHICAGO | MASTERS PROGRAM IN COMPUTER SCIENCE

Agenda

- ARM architecture/instructions overview
- ARM assembly program structure
- Saving registers and temporaries on the stack
- Generating and running an ARM executable
- Register Allocation

Code Generation



- After translating the code into an intermediate representation the next step is to either
  - Perform optimizations on the IR and convert into another IR to help the optimizer or continue translating the code in its current IR.
    - We will come back to this after code generation.
  - Translate the IR into a machine native code.
    - We will do this for ARM-64.

ARM System Architecture Overview

- ARM stands for Advanced RISC Machines
  - Originated in England in 1984, where it was mostly used in British educational systems
  - Late 80s and early 90s, ARM became the first commercial RISC processor and is now being used in many devices for its low power-energy usage and simple instruction set.
- Why use a RISC based architecture?
  - First, RISC stands for "Reduced Instruction Set Computer", contrasted with a CISC machine (Complex Instruction Set Computer, Intel).
  - RISC Advantages
    - Smaller die sizes
    - Shorter time to develop since it has a simple instruction set.
    - If optimized correctly, it can be more energy efficient.
  - RISC Disadvantages
    - Generally less code density than CISC
    - Cannot execute x86 code (i.e., intel) without some type of conversion. Apple has this conversion, which is known as Rosetta 2 for its new ARM Macs, in order to execute Intel-compiled programs.

ARM Program Structure

- An ARM assembly file contains a mixture of assembly directives, instructions, and labels:
  - Assembler directives are directions to the assembler to take some action or change a setting
    - They always begin with a period (“.”)
    - .text states the following lines will go inside the code segment of the virtual address space.
    - .arch armv8-a - states that this assembly file works on ARMV8-a architectures
  - The remaining code in an ARM program are ARM instructions and labels.

ARM Program Structure

- For the course compiler, the structure will be the following
  - Comments begin with // similar to Go and Golite

```
.arch armv8-a //Begin by specifying the architecture
<define all of the global variables>
.text // This specifies that following lines should be placed in the code segment
<define all of the functions in your program>
<define all labels used for interacting with the runtime system>
```

- Please note not all ARM64 assembly files are required to be structured in this way. For example, you can place global variable definitions at the end of the file

ARM Program Structure: Global Variables

- After the specifying the .arch armv8-a directive then you will want to define all global variables, with the .comm directive
  - Allocates storage in the data section in the virtual address space
  - Syntax: .comm name, size, alignment
  - Conveniently all variables will have size = 8, and alignment = 8 since our compiler represents everything as pointers or int64 integers.

Golite

```
var gVar1 int;
var gVar2 int;
//Assume we defined a Point struct
var gVar3 *Point;
```

Arm64 Assembly

```
.arch armv8-a
.comm   gVar1,8,8
.comm   gVar2,8,8
.comm   gVar3,8,8
```

- Please note not all ARM64 assembly files are required to be structured in this way. For example, you can place global variable definitions at the end of the file

ARM Program Structure: Code Segment

- After defining global variables, you must use the .text directive to state the remaining assembly code should go inside the code segment of the virtual address.

Arm64 Assembly

```
.arch armv8-a
.comm       gVar1,8,8
.comm       gVar2,8,8
.comm       gVar3,8,8
.text
// Below this line will contain all assembly code for a program
```

ARM Program Structure: Functions

- The remainder of the assembly file after .text will be functions along with their assembly code. Each must have the following structure:

```
            .type myFunc, %function
            .global myFunc
            .p2align      2  // All functions must be 16-bit aligned (i.e. 2 byte aligned)
myFunc:     // the start of the myFunc function
            //  myFunc function code goes here after the label
            .size myFunc, (. - myFunc) //end of the function always ends with .size
```

- .type name, type_description - directive sets the type of a symbol name to be either a function or an object. %function is the type_description for functions.
- .global name - directive makes the function accessible anywhere in the file or outside the file for external files.
- .size name, expression - directive is used to set the size associated with a function. This information helps the linker to exclude unneeded code and/or data when creating an executable file, and helps the debugger to keep track of where it is in the program.
  - (. - myFunc) - means subtract the location of the start of the function label (myFunc) to the current location of .size. This directive calculates how many bytes there are between the label and .size and is useful for a debugger/ linker.

ARM Program Structure: Functions structure

Golite

```
var gVar1 int;
var gVar2 int;
//Assume we defined a Point struct
var gVar3 *Point;
func fun2 (x int) {
    return x + 1;
}
func main() {
    fmt.Println(fun2(5));
}
```

Arm64 Assembly

```
        .arch armv8-a
        .comm   gVar1,8,8
        .comm.  gVar2,8,8
        .comm.  gVar3,8,8
        .text
        // Start the code segment (i.e., functions)
        .type   fun2, %function
        .global   fun2
        .p2align   2
fun2:
        //  fun2 code
        .size fun2, (. - fun2)
        .type   main, %function
        .global   main
        .p2align   2
main:
        //  main code
        .size main, (. - main)
```

ARM Architecture Instruction

- We will walk through the important instructions that is provided in the supplemental slides in this module directory
  - See “ARMv8_InstructionSetOverview.pdf”
  - You will not be using all of these instructions in this course.
  - **Important Slides**: 7-8, 12,14,17-18,25-26,30,32,40,43

General purpose and dedicated registers

- 31 general purpose + 2 dedicated registers, each 64-bit wide

same register

	GENERAL PURPOSE	DEDICATED	NOTES
Architectural names	R0, R1, R2, …, R30	SP, ZR	• names rarely used in practice
64-bit views	X0, X1, X2, …, X30	SP, XZR	• "x" stands for e**X**tended word • all 64 bits are "used"
32-bit views	W0, W1, W2, …, W30	WSP, WZR	• "w" stands for **W**ord • only **bottom** 32 bits are "used"

- Example: add two registers

INITIAL VALUES
X0=0x2222222222222222
X1=0x3333333333333333
X2=0xffffffffffffffff

ADD X2, X0, X1

AFTER 64-BIT ADD
X0=0x2222222222222222
X1=0x3333333333333333
X2=0x5555555555555555

All 64 bits of X2 are written

ADD W2, W0, W1

AFTER 32-BIT ADD
X0=0x2222222222222222
X1=0x3333333333333333
X2=0x0000000055555555

Upper 32 bits of X0, X1 are ignored.

Upper 32 bits of X2 are cleared on write

ARM®

Special dedicated registers

2 dedicated registers:

- **sp**, the stack pointer register: holds pointer to bottom of the stack
  - preferred register to access the stack
  - must be 16-bytes aligned

```
STR     W0, [SP, #4]     ; Stores W0 into the stack at address SP + 4.
ADD     SP, SP, #8       ; WARNING: SP is now "unusable": it is not aligned anymore!
STR     X0, [SP]         ; ERROR: cannot use unaligned SP!
```

- **zr:** the zero register
  - when used as source register it always returns the integer value zero.

```
MOV     W0, #0           ; W0 = 0
MOV     W0, WZR          ; W0 = 0, same effect as previous instruction
```

  - when used as a destination register it discards the value

```
SUBS    WZR, W10, W11    ; Does W10 – W11, set the flags and discard the result
CMP     W10, W11         ; Compare two numbers: CMP is an alias for the SUBS above
```

Two ways of writing the same instruction

ARM®

Register load/store

- **LDR**
  - Load data from an address into a register
- **STR**
  - Store data from a register to an address

```
LDR    X0, <addr>          ; Load from <addr> into X0
STR    X0, <addr>          ; Store contents of X0 to <addr>
```

- **By default, the size of the load/store is determined by the source/destination register name**
  - **Xn** will load/store 64 bits, **Wn** will load/store 32 bits
  - Instruction can be suffixed to force a smaller load/store size
    - **'B'** for byte, **'H'** for half-word, **'W'** for word
    - Result will be zero-extended by default, combine with the **'S'** suffix for sign-extension

```
LDRSB  X0, <addr>          ; Load byte from <addr> into X0 and sign-extend
STRH   W1, <addr>          ; Store half-word from W1 to <addr>
```


ARM®

Specifying the load/store address

- **Address to load/store from is a 64-bit base register plus an optional offset**

```
LDR    X0, [X1]                    ; Load from address held in X1
STR    X0, [X1]                    ; Store to address held in X1
```

- **Offset can be an immediate or a register**

```
LDR    X0, [X1, #8]                ; Load from address [X1 + 8 bytes]
LDR    X0, [X1, #-8]               ; Load with negative offset
LDR    X0, [X1, X2]                ; Load from address [X1 + X2]
```

- **A `Wn` register offset needs to be extended to 64 bits**

```
LDR    X0, [X1, W2, SXTW]          ; Sign-extend offset in W2
LDR    X0, [X1, W2, UXTW]          ; Zero-extend offset in W2
```

- **Both `Xn` and `Wn` register offsets can include an optional left-shift**

```
LDR    X0, [X1, W2, UXTW #2]       ; Zero-extend offset in W2 & left-shift by 2
LDR    X0, [X1, X2, LSL #2]        ; Left-shift offset in X2 by 2
```


ARM®

Register pair load/store

- **New Load Pair and Store Pair instructions**
  - Support both integer and scalar FP / SIMD registers
  - Both source/destination registers must be the same width

```
LDP    W3, W7, [X0]              ; [X0] => W3, [X0 + 4 bytes] => W7
STP    Q0, Q1, [X4]              ; Q0 => [X4], Q1 => [X4 + 16 bytes]
```

- **No Load Multiple, Store Multiple, or `PUSH` / `POP` instructions in AArch64**
  - Construct these operations using **`STP`** and **`LDP`** instructions

```
STP    X0, X1, [SP, #-16]!       ; Push X0 and X1 onto the stack
LDP    X0, X1, [SP], #16         ; Pop X0 and X1 from the stack
```


ARM®

Using the PC

- **There are variants of `LDR` to load `PC` relative data**
  - Use a label operand rather than a 64-bit base address register
  - Linker generates a `PC` relative load from the address of the label in the executable image

```
LDR     X0, label              ; Load value at <label>
```

- **Assemblers may support a "Load (immediate)" pseudo-instruction**
  - Creates a `PC` relative load, and a literal pool containing the value to be loaded

```
LDR     X0, =imm               ; Load from literal containing imm
```

- **Obtaining the address of a label**
  - `PC` relative loads and `ADR` are limited in range to ±1MB, whereas `ADRP` has range ±4GB

```
LDR     X0, =label             ; Load address of label from literal pool
ADR     X0, label              ; Calculate address of label (PC relative)
ADR     X0, .                  ; Get current PC (address of ADR instruction)
ADRP    X0, label              ; Calculate address of 4KB page containing label
```


ARM®

Multiplication and division

- Regular 32-bit and 64-bit multiplication:
  - `MUL Rd, Rn, Rm` → `Rd = Rn*Rm` (alias of `MADD: Ra = ZR`)
  - `MADD Rd, Rn, Rm, Ra` → `Rd = Ra + Rn*Rm`
  - `MSUB Rd, Rn, Rm, Ra` → `Rd = Ra - Rn*Rm`
  - `MNEG Rd, Rn, Rm` → `Rd = -Rn*Rm` (alias of `MSUB: Ra = ZR`)
- Long result multiplication: 32-bit source registers, 64-bit destination register.
  - Signed variants: `SMULL`, `SMADDL`, `SMSUBL`, `SMNEGL`
  - Unsigned variants: `UMULL`, `UMADDL`, `UMSUBL`, `UMNEGL`
  - Upper 64 bits in 128-bit multiplication result: `UMULH`, `SMULH`
- Signed and unsigned 32-bit and 64-bit division
  - `SDIV` / `UDIV Rd, Rm, Rn` → `Rd = Rn/Rm`
  - Division by 0 returns 0 (with no exception)
  - `MAXNEG` integer divided by -1 overflows (returns `MAXNEG`)

ARM®

Branches

- `B <offset>`
  - `PC` relative branch ±128 MB
  - Conditional version `B.cond` (covered later) has ±1 MB range
- `BL <offset>`
  - Similar to `B` (branch range ±128 MB) but also stores return address in `LR` (`X30`), hinting that this is a function call
  - No conditional version
- `BR Xm`
  - Absolute branch to address in `Xm`
- `BLR Xm`
  - Similar to `BR`, but also stores return address in `LR` (`X30`), hinting that this is a function call
- `RET Xm` or simply `RET`
  - Similar to `BR`, but also hints that this is a function return
  - Use `LR` (`X30`) if register is omitted, but can use other register

ARM®

Conditional branches

- `B.cond`
  - Branch to label if condition code evaluates to true

```
CMP     X0, #5
B.EQ    label                   ; Branch to label if (X0 == #5)
```

- `CBZ` / `CBNZ`
  - Branch to label if operand register is equal to zero (**`CBZ`**) or not equal to zero (**`CBNZ`**)

```
CBZ     X0, label               ; Branch to label if (X0 == #0)
CBNZ    W0, label               ; Branch to label if (W0 != #0)
```

- `TBZ` / `TBNZ`
  - Branch to a label if a specific bit in the operand register is set (**`TBNZ`**) or cleared (**`TBZ`**)

```
TBZ     W0, #20, label          ; Branch to label if (W0[20] == #0b0)
TBNZ    X0, #50, label          ; Branch to label if (X0[50] == #0b1)
```


ARM®

Example: Condition execution

C Source Code:

```
if (a == 0)
{
        y = y + 1;
}
else
{
        y = y - 1;
}
```

A64 Conditional Branching:

```
        CMP     W0, #0
        B.NE    else
        ADD     W1, W1, #1
        B       end
else:
        SUB     W1, W1, #1
end:
```

A32 Conditional Execution:

```
CMP     R0, #0
ADDEQ   R1, R1, #1
SUBNE   R1, R1, #1
```

A64 Conditional Operations:

```
        CMP     W0, #0
        SUB     W2, W1, #1
        CSINC   W1, W2, W1, NE
```

```
CSINC W0, W1, W2, cond
W0 = (cond) ? W1 : W2 + 1
```


ARM®

AAPCS64: Role of integer registers

Register	Alternative name	Role
R0		Return value (for integers and pointers)
R0 … R7		Arguments in function calls (for integers and pointers)
R8		Indirect result location register. Used in C++ for returning non-trivial objects (set by the caller).
R9 … R15		Temporary registers (trashed across calls)
R16, R17	IP0, IP1	The intra-procedure-call temporary registers. The linker may use these in PLT code. Can be used as temporary registers between calls
R18		Platform register
R19 … R28		Callee-saved registers: register preserved across calls
R29	FP	Frame pointer. Copy of SP before function stack allocation
R30	LR	Link register. BL and BLR instructions save return address in it
SP		Stack pointer

ARM®

Condition codes

Condition Code	Description	Flags Tested
EQ	Equal	Z == 1
NE	Not Equal	Z == 0
CS / HS	Unsigned Higher or Same	C == 1
CC / LO	Unsigned Lower	C == 0
MI	Minus	N == 1
PL	Positive or Zero	N == 0
VS	Overflow	V == 1
VC	No Overflow	V == 0
HI	Unsigned Higher	C == 1 && Z == 0
LS	Unsigned Lower or Same	C == 0 && Z == 1
GE	Greater Than or Equal	N == V
LT	Less Than	N != V
GT	Greater Than	Z == 0 && N == V
LE	Less Than or Equal	Z == 1 || N != V
AL	Always	--

NZCV → Negative, Zero, Carry, Overflow

ARM®

ARM Control Flow

- To implement the control flow constructs (i.e., loops, conditionals, etc.) you will need to use a mixture of labels to represent their bodies and conditional expressions and branching operations to jump to those labels.
  - We are using the following syntax to represent implementing the structure of the control flow constructs
    - br_{false} - means jump when the condition of construct is false.
    - br_{true} - means jump when the condition of construct is true.

Negating Expressions with Equality, Relational and Logical Operators

Expression	Negation of Expression
exprLeft == exprRight	exprLeft != exprRight
exprLeft != exprRight	exprLeft == exprRight
exprLeft > exprRight	exprLeft <= exprRight
exprLeft >= exprRight	exprLeft < exprRight
exprLeft < exprRight	exprLeft >= exprRight
exprLeft <= exprRight	exprLeft > exprRight
exprLeft && exprRight	! (exprLeft && exprRight) ==> !exprLeft || !expRight
exprLeft || exprRight	! (exprLeft || exprRight) ==> !exprLeft && !expRight

ARM: Control-flow (for loop)

- Syntax: for (cond) stm
- ARM Structure implementation:

```
            b testCondL
loopBodyL:
            < ARM code for evaluating stm  >
testCondL:
            < ARM code for evaluating cond >
            brtrue  loopBodyL
```

- Example

```
for (x < 10) {
   x = x + 1;
   y = x *  2;
}
```

→

Assumptions: x is assigned to x0, y is assigned to x1

```
            b condLabel1
loopBody1:
            add x0, x0,#1
            mul x1, x0,#2
condLabel1:
            mov x2, #10
            cmp X2,X0
            b.lt loopBody1
```

- **Important**: In generated assembly code, we will need to have unique labels for each loop, conditional statement, etc.

ARM Control-flow (if statement)

- Syntax: if (cond) stmt
- ARM Structure implementation:

```
        < ARM code for evaluating cond >
        brfalse skipIf
        < ARM code for evaluating stmt >
skipIf:
```

- Example

```
if (x < 10) {
   x = x + 1;
   y = x *  2;
}
X = 200;
```

→

```
Assumptions: x is assigned to x0, y is assigned to x1

        mov x2,#10
        cmp x2,x0
        b.ge skip1      # Notice this is the false branch instruction!
        add x0,x0,#1
        mul x1,x0,#2
skip1:
        mov x0,#200
```

- **Important**: In generated assembly code, we will need to have unique labels for each loop, conditional statement, etc.

ARM: Control-flow (if-else statement)

- Syntax: if (cond) stmt1 else stmt2
- ARM Structure implementation:

```
                < ARM code for evaluating cond >
                 br_false elseLabel
                < ARM code for evaluating stmt1 >
                 b done
elseLabel:
               < ARM code for evaluating stmt2 >
done:
```

- **Important**: In generated assembly code, we will need to have unique labels for each loop, conditional statement, etc.

Loading and Storing Global Variables

- Loading and storing global variable values requires usage of the adrp and add instructions.

```
adrp  target_register, global_var_name
add  target_register, target_register, :lo12:global_var_name
//target_register now has the address of global_var_name from the data section.
```

Golite

```
var gVar1 int;
var gVar2 int;
func main() {
   gVar2 = 2000 + gVar1;
}
```

ARM

```
        .arch armv8-a
        .comm    gVar1,8,8
        .comm.   gVar2,8,8
        .text
        … // removing the function setup code
main:
        … // Not showing startup code
        adrp  x8, gVar1
        add   x8, x8, :lo12:gVar1       // x8 = &gVar1
        ldr     x8, [x8]                // x8 = gVar1 ( 0 initially)
        mov  x9, #20000                 // x9 = 2000
        adrp  x10, gVar2
        add   x10, x10, :lo12:gVar2     // x10 = &gVar2
        add.  x8, x9, x8                // x8 = x9 + x8 = 2000 + gVar1
        str   x8, [x10]                 // gVar2 = x8
```

Labels in Functions

- All labels defined within functions must begin with a "." followed by the label name.
  - Function names do not begin with a period

```
        .arch armv8-a
        .type   main, %function
        .global   main
        .p2align.   2
main:
        b .L1
.L:
        add r0,r1,r2
        //..
        .size main, (. - main)
```


THE UNIVERSITY OF CHICAGO | MASTERS PROGRAM IN COMPUTER SCIENCE

ARM64 Memory Layout

- The terminology size or length means how many bytes does a value of a specific type take up in memory
  - Golite: 8 bytes (integers), 8 bytes (pointers to structs)
- The term alignment is an indication that a specific variable needs to be located at a memory address that is a multiple of the alignment value.
  - For example, the stack pointer for ARM64 must be 16-bytes aligned. This means the stack pointer must be located on a memory address that is a multiple of 16.

Procedure Call Standard ARM64

- Passing arguments to functions
  - The first 8 arguments to a function must be passed to registers.

Register	Argument
x0	arg1
x1	arg2
…	…
x7	arg8

  - Additional arguments are passed on the stack. Arguments are pushed right-to-left (RTL), which means next arguments are closer to current sp.

```
func foo(…, a9 int, a10 int)
                 |        |        | ...  |          Hi
                 |        +——>     | a10 |           |
                 +————————>        | a9  | <-SP      |
                                   +-----+           v
                                   | ….  |           Lo
```

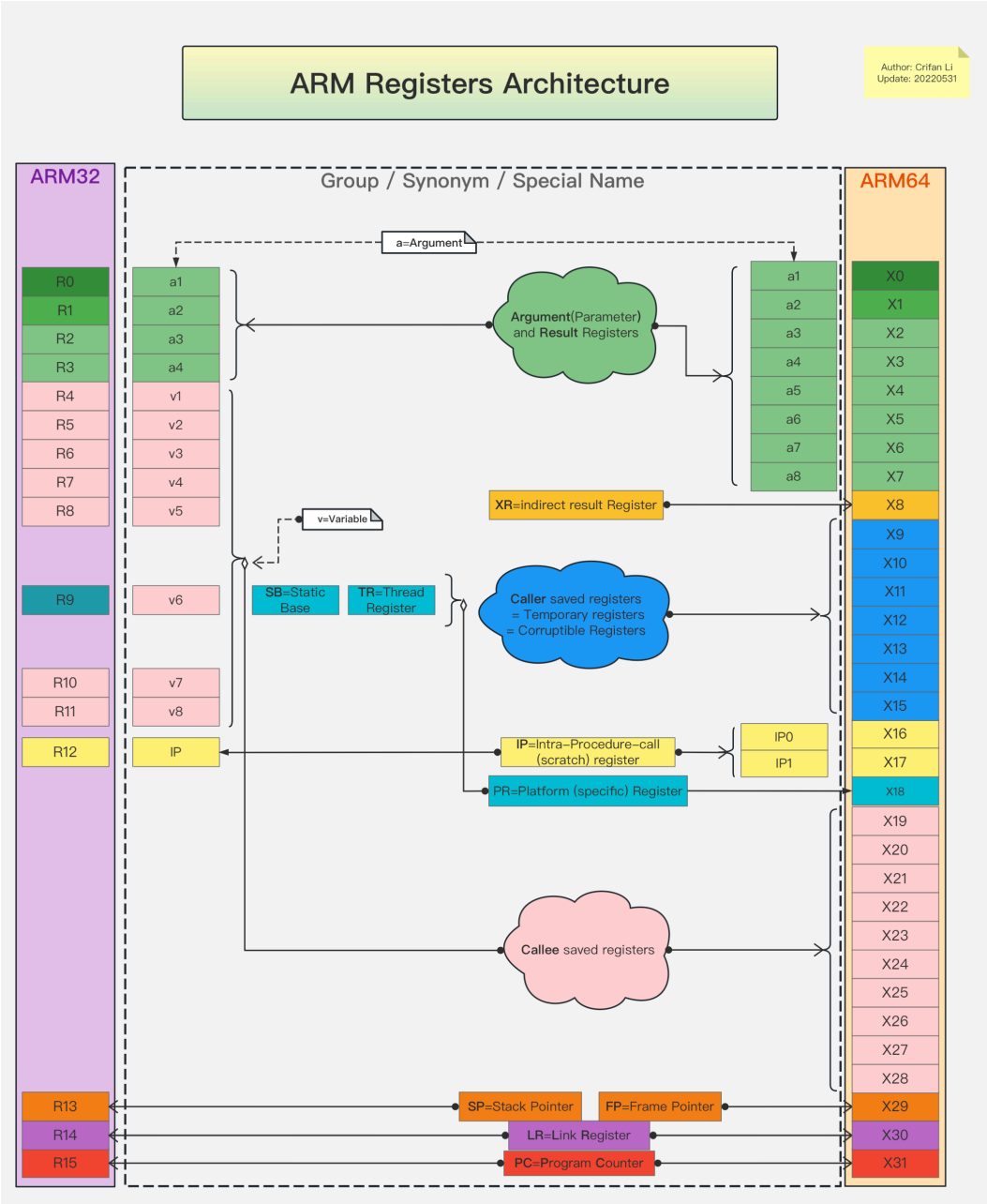

THE UNIVERSITY OF CHICAGO | MASTERS PROGRAM IN COMPUTER SCIENCE

Procedure Call Standard ARM64

- All return values for functions that return a value are stored in register 0.

Register	Size
x0	64 bits

- Registers R0-R15 are known as caller-saved temporary registers
  - From ARM: *"If the caller requires the values in any of these registers to be preserved across a call to another function, the caller must save the affected registers in its own stack frame. They can be modified by the called subroutine without the need to save and restore them before returning to the caller."*
  - Make sure to save these to the stack **BEFORE** calling the function.
- Registers R19-R29 are know as callee-saved registers
  - From ARM: *"These registers are saved in the callee frame. They can be modified by the called subroutine as long as they are saved and restored before returning."*
  - This means that if the callee function is required to use registers pass 19 that before you start executing the callee function you must first save the current values of these registers in the callee frame and before exiting you must restore the values
- Do not use SP or R30 (Link Register).



Cite: (Crifan Li)https://www.processon.com/view/link/6295e1b80791294815547608

Procedure Call Standard ARM64

- If a program is using more than 29 registers than you will have to spill registers (i.e., store) the remaining registers onto the stack frame.
  - You will know how many registers to spill after register allocation.
- ARM64 Stack Frame
  - Fully descending: sp points to the last used location (i.e., it points to the last item added to the stack).
  - Descending: stack grows downwards
  - sp must be 16byte aligned when used to access memory for r/w

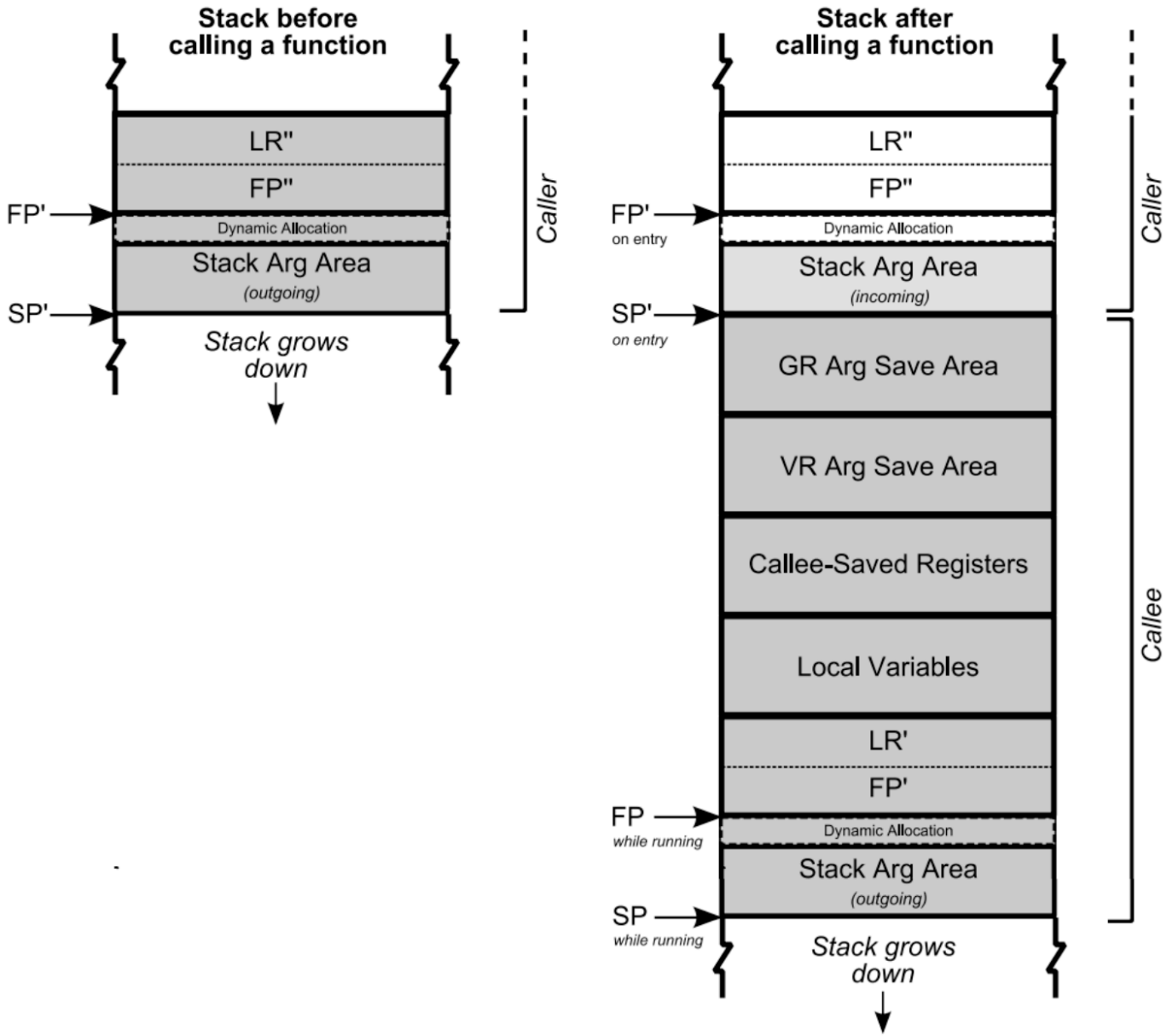
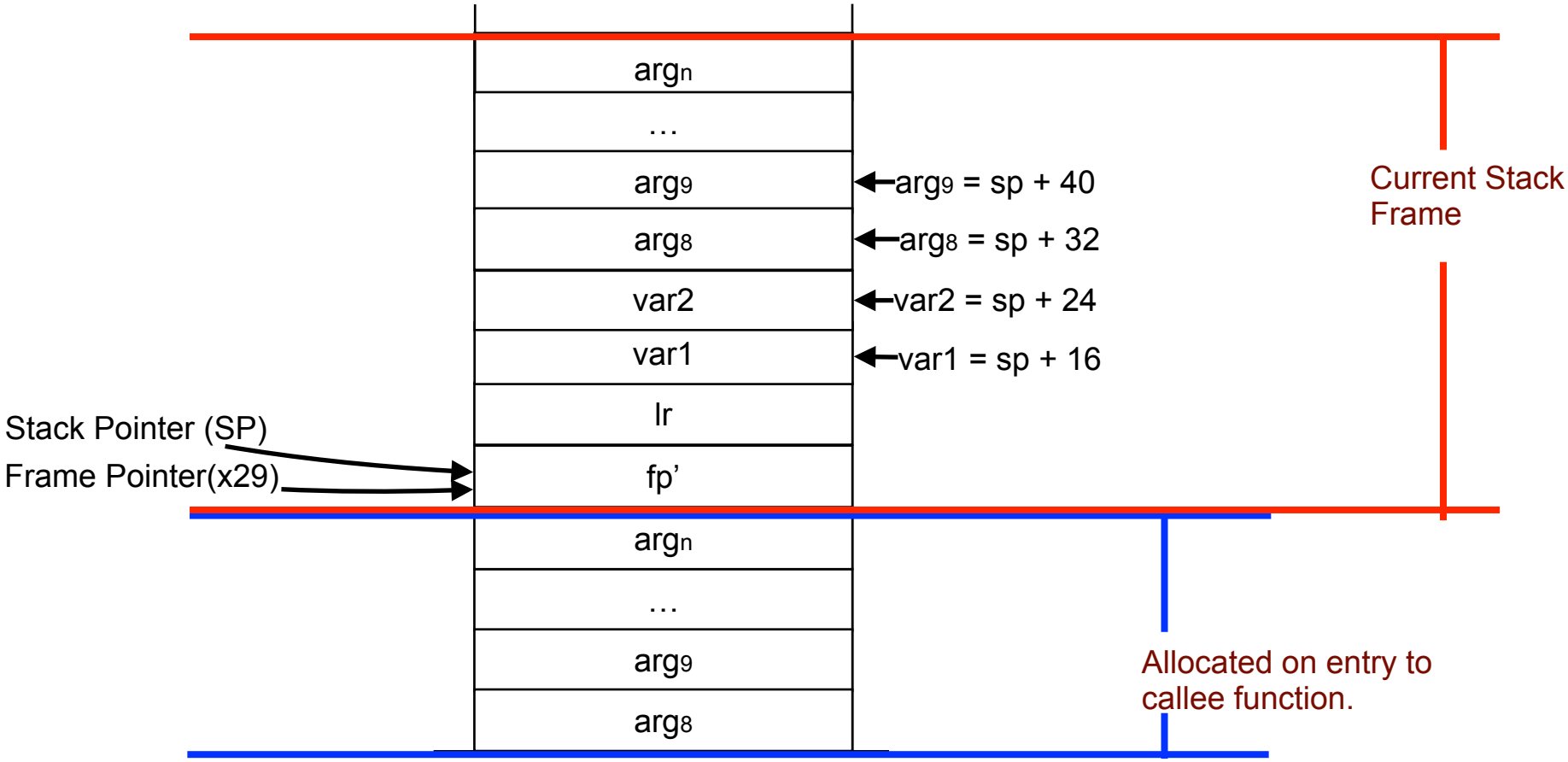


Diagram Cite: *Procedure Call Standard for the Arm® 64-bit Architecture (AArch64)* 2020Q4 Date of Issue: 21 December 2020

Accessing the Stack on ARM64

- Assuming no temps, saved-registers, etc. with only two local variables. Here's how you would access local and arguments.



Function Prologue & Epilogue

- When entering and exiting a function you must push or pop off the frame pointer and link register.

- Prologue - must be performed at the start of the code within the callee function

```
sub sp, sp, 16          // Decrement SP to allocate space for previous frame's LR and FP
stp x29, x30, [sp]      // [sp] = x29; [sp + 8] = x30
mov x29, sp             // FP points to frame record
// You'll need to update sp again to make space for the entire incoming frame or you can
// from the beginning.
```

- Epilogue

```
ldp x29, x30, [sp]      // x29 = [sp]; x30 = [sp + 8]
add sp, sp, 16          // "deallocate" the space for the FP and LR
ret
// You will need to make sure before this code or after that you increment sp to remove the
// space for the callee frame.
```

Runtime System Calls

- We will be using the C runtime system to make it easer to
  - Allocate memory using malloc
  - Deallocate memory using free
  - Reading in an integer using scanf
  - Printing to the console using printf

- All these functions are linked in later when creating the executable. You do not need to define them everywhere in your file.

Malloc runtime call

- In Golite, you will need to allocate structs values on the heap. Therefore you will need to use malloc. Before calling malloc you must store the number of bytes you wish to allocate into r0.
  - The address to the allocated memory will be stored back into r0 when the function returns.

```
type Point struct {
    x int;
    y int;
};
func main() {
   var p *Point;
   p = new Point;
}
```

```
// I'm not showing all the code here just what's needed for allocation purposes

// The size of Point is 16 bytes. 8 bytes for each field.
mov x0, #16
bl malloc
// Now x8 has the address to the allocated memory for the Point since malloc stores the return value in r0.
mov x8, x0
```

Free runtime call

- In Golite, you must free all memory you allocate by using the free function to deallocate the memory at an address.
  - The address to deallocate needs to be store in r0.

```
type Point struct {
    x int;
    y int;
};
func main() {
   var p *Point;
   p = new Point;
   delete p;
}
```

```
// I'm not showing all the code here just what's needed for allocation purposes

// The size of Point is 16 bytes. 8 bytes for each field.
mov x0, #16
bl malloc
// Now x8 has the address to the allocated memory for the Point since malloc stores the return value in r0.
mov x8, x0
…
mov x0, x8 // move the address to deallocate into x0
bl free  // call free to deallocate the memory
```

Printing call

- Similar to Printf in Go, you need to specify the formatter string and the variables to print. These are one or more separate arguments.
  - The first argument must be the format string and the address must be placed in r0. Similar to LLVM IR, each format string should have a label associated with it. You need to specify the .asciz and .size directors that represents the string in ascii format and the size plus space for the null character.
  - Similar to global variables always use adrp and add to get the address of the format string.
  - Key things: x0 must always be the address of the format string (e.g. .FMTSTR1), and the registers (and possibly stack) hold the value of the expressions to print.

Define the following labels as the very last labels in the file.

```
.FMTSTR1:
        .asciz      "x=%ld"
        .size       .PRINT, 6
```

```
x = 4;
printf("x=%d",x);
```

```
// I'm not showing all the code here just what's needed for printing purposes
adrp        x10, .FMTSTR1
add         x10, x10, :lo12:.FMTSTR1
mov x0, x10
mov x1, x4 // Assume x is assigned to x4
bl printf
```

Reading in an Integer

- Similar to Scanf in Go, you need to specify the address of the variable to place the read in value along with the format string for reading in an int64. These are two separate arguments.
  - The first argument must be the format string and the address must be placed in r0. Use the following code to always create the format string. This will not change for any program.
  - Similar to global variables always use adrp and add to get the address of the format string.
  - **Scanf will store the value read in into the address of x1. You will need to perform a load operation to place that value into a register before using it!**
  - Key things: r0 must always be the address of the either .READ. r1 holds the **address of the variable** to place the read in value.

Scanf format string

```
.READ:
        .asciz      "%ld"
        .size       .READ, 4
```

```
var x int;
x = scan;
```

```
// I'm not showing all the code here just what's needed for printing purposes
adrp        x11, .READ
add         x11, x11, :lo12:.READ
mov x0, x11
add  x1,sp, #16  //Assume x is the first local variable on the frame
bl scanf
```

Running an ARM Program

- We will use the “clang” compiler’s (i.e., a popular C compiler) assembler and linker to create the executable for our Golite programs.
  - We are doing this since we are using the runtime system of the clang.
- To run code you will need to login to the ARM CS server with your CS credentials

wing.cs.uchicago.edu

- I recommend you login using the VS Studio Code method described on the CS student resources site:
  - https://uchicago-cs.github.io/student-resource-guide/vscode/ssh.html
- Your compiler can produce .s (i.e., an assembly file), and then produce the executable and run it as follows:
-

```
$: gcc -o PROGRAM_NAME GOLITE_PROGRAM_FILE_NAME.s
$: ./PROGRAM_NAME
```

Demo: Writing and Running a simple Arm Program

Register Allocation

- LLVM uses an infinite number of registers when generating code but when translating too assembly we only have a finite number of registers to use.
- Thus, we need to have use a register allocation scheme
  - A process that assigned variables and temporaries to registers and manages the transfer of data in and out of registers.
- For Golite compiler, we are first going to use a stack allocation approach.
  - This approach is NEVER used in production compilers; however, it's a simple approach to understanding the generation of IR code to assembly.
  - A better approach is to use a graph-coloring register allocation.

Stack Based Allocator

- The naive register allocation approach is as follows:
  - Allocate enough space on the stack for all arguments, locals, and temp variables.
  - You must still adhere the ARM 64 Procedure Call standard (e.g., x0-x7 will always hold the first arguments).
  - There will be a large number of load/store calls based on always accessing and updating the stack.
- Note: This scheme will only work on small programs since this scheme could potentially cause a stack overflow.

Stack Based Approach: Example

Source Code

```
func foo(a int, b int) int
{
   var c int;
   c = b + a * 2;
   return c;
}
```

LLVM IR

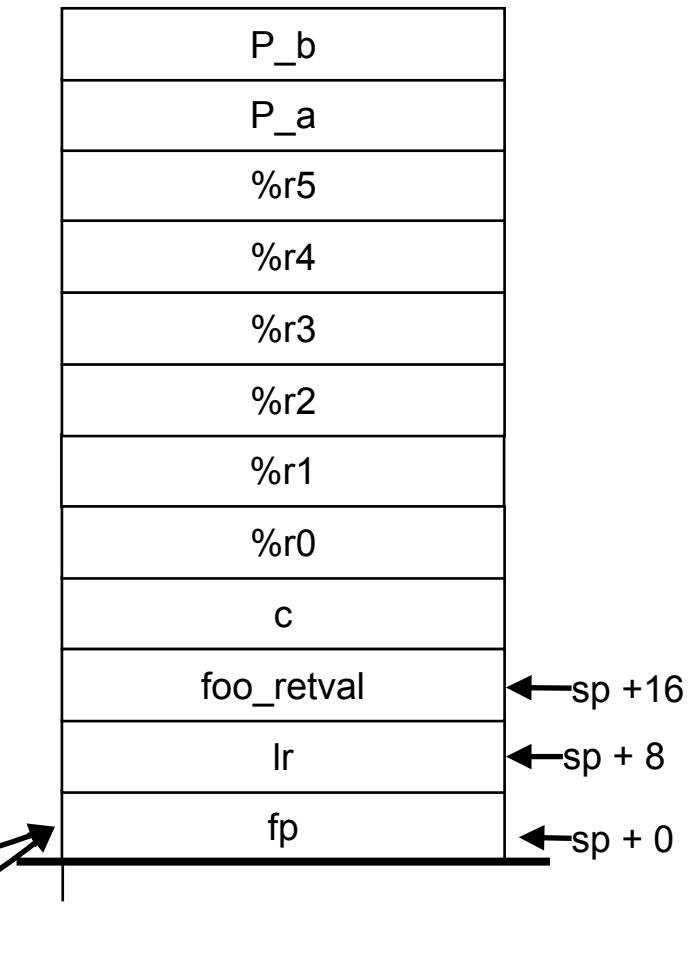
```
define i64 @foo (i64 %a, i64 %b)
{
L1:
   %foo_retval = alloca i64
   %P_a = alloca i64
   %P_b = alloca i64
   %c = alloca i64
   store i64 %a, i64* %P_a
   store i64 %b, i64* %P_b
    %r0 = load i64, i64* %P_a
    %r1 = mul i64 %r0, 2
    %r2 = load i64, i64* %P_b
    %r3 = add i64 %r2, %r1
    store i64 %r3, i64* %c
    br label %L2
L2:
   %r4 = load i64, i64* c
   store i64 %r4, i64* %foo_retval
   %r5 = load i64, i64* %foo_retval
   ret i64 %r5
}
```

Stack Based Approach: Example

LLVM IR

```
define i64 @foo (i64 %a, i64 %b)
{
L1:
   %foo_retval = alloca i64
   %P_a = alloca i64
   %P_b = alloca i64
   %c = alloca i64
   store i64 %a, i64* %P_a
   store i64 %b, i64* %P_b
    %r0 = load i64, i64* %P_a
    %r1 = mul i64 %r0, 2
    %r2 = load i64, i64* %P_b
    %r3 = add i64 %r2, %r1
    store i64 %r3, i64* %c
    br label %L2
L2:
   %r4 = load i64, i64* c
   store i64 %r4, i64* %foo_retval
   %r5 = load i64, i64* %foo_retval
   ret i64 %r5
}
```

Stack Frame (foo)



Variables Symbol Table (foo)

Variable	Location
a	x0
b	x1
foo_retval	16
P_a	80
P_b	88
c	24
r0	32
r1	40
r2	48
r3	56
r4	64
r5	72

Stack Based Approach: Example

LLVM IR

```
define i64 @foo (i64 %a, i64 %b)
{
L1:
    %foo_retval = alloca i64
    %P_a = alloca i64
    %P_b = alloca i64
    %c = alloca i64
    store i64 %a, i64* %P_a
    store i64 %b, i64* %P_b
     %r0 = load i64, i64* %P_a
     %r1 = mul i64 %r0, 2
     %r2 = load i64, i64* %P_b
     %r3 = add i64 %r2, %r1
     store i64 %r3, i64* %c
     br label %L2
L2:
    %r4 = load i64, i64* c
    store i64 %r4, i64* %foo_retval
    %r5 = load i64, i64* %foo_retval
    ret i64 %r5
}
```

Variables Symbol Table (foo)

Variable	Location
a	x0
b	x1
foo_retval	16
P_a	80
P_b	88
c	24
r0	32
r1	40
r2	48
r3	56
r4	64
r5	72

ARM

```
    .type foo, %function
    .global foo
    .p2align 2

foo:
.L1:
    // Prologue
    sub sp, sp, #96
    stp x29, x30, [sp]
    mov x29, sp
```

SP = 10 variables * 8 bytes + FP (8 Bytes) + LR (8 bytes) = 10 * 8 + 8 + 8 = 96
Make sure it is a multiple of 16. If not round up to the next multiple of 16!

Stack Based Approach: Example

LLVM IR

```
define i64 @foo (i64 %a, i64 %b)
{
L1:
    %foo_retval = alloca i64
    %P_a = alloca i64
    %P_b = alloca i64
    %c = alloca i64
    store i64 %a, i64* %P_a
    store i64 %b, i64* %P_b
     %r0 = load i64, i64* %P_a
     %r1 = mul i64 %r0, 2
     %r2 = load i64, i64* %P_b
     %r3 = add i64 %r2, %r1
     store i64 %r3, i64* %c
     br label %L2
L2:
    %r4 = load i64, i64* c
    store i64 %r4, i64* %foo_retval
    %r5 = load i64, i64* %foo_retval
    ret i64 %r5
}
```

Variables Symbol Table (foo)

Variable	Location
a	x0
b	x1
foo_retval	16
P_a	80
P_b	88
c	24
r0	32
r1	40
r2	48
r3	56
r4	64
r5	72

ARM

```
    .type foo, %function
    .global foo
    .p2align 2

foo:
.L1:
    // Prologue
    sub sp, sp, #96
    stp x29, x30, [sp]
    mov x29, sp
    str x0, [sp,#80]
    str x1, [sp,#88]
```

Stack Based Approach: Example

LLVM IR

```
define i64 @foo (i64 %a, i64 %b)
{
L1:
    %foo_retval = alloca i64
    %P_a = alloca i64
    %P_b = alloca i64
    %c = alloca i64
    store i64 %a, i64* %P_a
    store i64 %b, i64* %P_b
     %r0 = load i64, i64* %P_a
     %r1 = mul i64 %r0, 2
     %r2 = load i64, i64* %P_b
     %r3 = add i64 %r2, %r1
     store i64 %r3, i64* %c
     br label %L2
L2:
    %r4 = load i64, i64* c
    store i64 %r4, i64* %foo_retval
    %r5 = load i64, i64* %foo_retval
    ret i64 %r5
}
```

Variables Symbol Table (foo)

Variable	Location
a	x0
b	x1
foo_retval	16
P_a	80
P_b	88
c	24
r0	32
r1	40
r2	48
r3	56
r4	64
r5	72

ARM

```
    .type foo, %function
    .global foo
    .p2align 2

foo:
.L1:
    // Prologue
    sub sp, sp, #96
    stp x29, x30, [sp]
    mov x29, sp
    str x0, [sp,#80]
    str x1, [sp,#88]
    ldr x2, [sp,#80]
    str x2, [sp,#32]
```

Stack Based Approach: Example

LLVM IR

```
define i64 @foo (i64 %a, i64 %b)
{
L1:
    %foo_retval = alloca i64
    %P_a = alloca i64
    %P_b = alloca i64
    %c = alloca i64
    store i64 %a, i64* %P_a
    store i64 %b, i64* %P_b
     %r0 = load i64, i64* %P_a
     %r1 = mul i64 %r0, 2
     %r2 = load i64, i64* %P_b
     %r3 = add i64 %r2, %r1
     store i64 %r3, i64* %c
     br label %L2
L2:
    %r4 = load i64, i64* c
    store i64 %r4, i64* %foo_retval
    %r5 = load i64, i64* %foo_retval
    ret i64 %r5
}
```

Variables Symbol Table (foo)

Variable	Location
a	x0
b	x1
foo_retval	16
P_a	80
P_b	88
c	24
r0	32
r1	40
r2	48
r3	56
r4	64
r5	72

ARM

```
    .type foo, %function
    .global foo
    .p2align 2

foo:
.L1:
    // Prologue
    sub sp, sp, #96
    stp x29, x30, [sp]
    mov x29, sp
    str x0, [sp,#80]
    str x1, [sp,#88]
    ldr x2, [sp,#80]
    str x2, [sp,#32]
    ldr x3, [sp,#32]
    mov x2,2
    mul x4, x3, x2
    str x4, [sp,#40]
```

Analysis of Naive Register Allocation

- Disadvantage: Horribly inefficient
  - Generates way too many unnecessary load and store instructions
  - Wastes space on temporaries that can be stored in unused registers
  - Memory loads and stores are a hinderance to performance to this version will result in slower programs
  - Never used in production compilers
- Advantage: Simplicity
  - Can easily translate each IR instruction directly to assembly.
  - Never need to worry about running out of registers, function calls or special-purpose registers.
  - From a pedagogy perspective, it's great for quickly implementing code generation for a prototype language such as Golite.
- Recommend using a graph-coloring register allocation.

LLVM to ARM Assembly

- Similar to AST to LLVM, one way to implement the translation from LLVM to ARM is to define a method for each LLVM instruction that will translate its LLVM representation to assembly. For example, here's how the Add ILOC instruction could look like:

```
func (instr *Add) TranslateToAssembly(tables * st.SymbolTables)
[]codegen.Instruction {

}
```

- Overall, you should define a codegen package that contains a function that translates your function fragments into a code generation function

```
func TranslateToAssembly([]*BasicBlock, tables *
st.SymbolTables) ... ... {

}
```